

Q)1 Introduction to Database Creation

Objective: Set up and create a database using MySQL/PostgreSQL. Task: Entities and Relationships: • Student (StudentID, Name, Email, Age, Address) • Course (CourseID, CourseName, Credits, InstructorID) • Instructor (InstructorID, Name, Email, Department) • Enrollment (EnrollmentID, StudentID, CourseID, EnrollmentDate) Relationships: • A student can enroll in multiple courses. • A course can have multiple students. • An instructor teaches one or more courses.

```
CREATE DATABASE IF NOT EXISTS CollegeDB;
```

```
USE CollegeDB;
```

```
DROP TABLE IF EXISTS Enrollment;
```

```
DROP TABLE IF EXISTS Course;
```

```
DROP TABLE IF EXISTS Instructor;
```

```
DROP TABLE IF EXISTS Student;
```

```
-- Student Table
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Age INT,  
    Address VARCHAR(100)  
);
```

```
-- Instructor Table
```

```
CREATE TABLE Instructor (  
    InstructorID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Department VARCHAR(50)  
);
```

```
-- Course Table
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    Credits INT,  
    InstructorID INT,
```

```

FOREIGN KEY (InstructorID)
REFERENCES Instructor(InstructorID)
);

-- Enrollment Table
CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID)
REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID)
REFERENCES Course(CourseID)
);

-- Insert into Student
INSERT INTO Student VALUES
(1, 'Arjun', 'arjun@gmail.com', 21, 'Pune'),
(2, 'Rahul', 'rahul@gmail.com', 22, 'Mumbai'),
(3, 'Sneha', 'sneha@gmail.com', 20, 'Delhi');

-- Insert into Instructor
INSERT INTO Instructor VALUES
(101, 'Dr. Sharma', 'sharma@gmail.com', 'Computer Science'),
(102, 'Dr. Mehta', 'mehta@gmail.com', 'Information Technology');

-- Insert into Course
INSERT INTO Course VALUES
(201, 'Database Systems', 4, 101),
(202, 'Python Programming', 3, 102),
(203, 'Artificial Intelligence', 4, 101);

-- Insert into Enrollment
INSERT INTO Enrollment VALUES
(1, 1, 201, '2026-01-10'),
(2, 2, 202, '2026-01-11'),

```

```
(3, 3, 203, '2026-01-12'),  
(4, 1, 202, '2026-01-13');  
  
-- Display Tables  
  
SHOW TABLES;  
  
-- Display Structure  
  
DESC Student;  
  
-- Display Records  
  
SELECT * FROM Student;  
  
SELECT * FROM Instructor;  
  
SELECT * FROM Course;  
  
SELECT * FROM Enrollment;
```

Q2) Views and Indexing

Optimizing a Student Course Management System with Views and Indexing for Faster Query Performance.

Objective: Understand views and indexing for performance optimization. Task:

- Create views for easy reporting on students and course details.
- Update and delete records using views where applicable.
- Use indexing to speed up searches and measure query performance improvements.

```
-- Create Database  
  
CREATE DATABASE StudentManagement;  
  
  
-- Use Database  
  
USE StudentManagement;  
  
  
-- Create Student Table  
  
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Age INT
```

```
);
```

```
-- Create Course Table
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    Credits INT  
);
```

```
-- Insert Records into Student
```

```
INSERT INTO Student VALUES  
(1, 'Arjun', 'arjun@gmail.com', 21),  
(2, 'Rahul', 'rahul@gmail.com', 22),  
(3, 'Sneha', 'sneha@gmail.com', 20);
```

```
-- Insert Records into Course
```

```
INSERT INTO Course VALUES  
(101, 'Database Systems', 4),  
(102, 'Python Programming', 3),  
(103, 'Machine Learning', 4);
```

```
-- Create View for Student Details
```

```
CREATE VIEW StudentView AS  
SELECT StudentID, Name, Email, Age  
FROM Student;
```

```
-- Display Student View
```

```
SELECT * FROM StudentView;
```

```
-- Create View for Course Details
```

```
CREATE VIEW CourseView AS  
SELECT CourseID, CourseName, Credits
```

FROM Course;

-- Display Course View

SELECT * FROM CourseView;

-- Update Record using View

UPDATE StudentView

SET Age = 23

WHERE StudentID = 1;

-- Display Updated Record

SELECT * FROM StudentView;

-- Delete Record using View

DELETE FROM StudentView

WHERE StudentID = 3;

-- Display Records after Delete

SELECT * FROM StudentView;

-- Create Index on Student Name

CREATE INDEX idx_student_name

ON Student(Name);

-- Create Index on Course Name

CREATE INDEX idx_course_name

ON Course(CourseName);

-- Check Query Performance

EXPLAIN SELECT * FROM Student

WHERE Name = 'Arjun';

```
EXPLAIN SELECT * FROM Course
WHERE CourseName = 'Database Systems';
```

Q3) SQL Queries with Filters and Sorting ‘

Advanced Querying in a Student Course Management System Using Filters, Sorting, and Nested Subqueries Objective: Use SQL filters and sorting clauses. Task: • Filtering Data with WHERE (Find students above a certain age, filter by instructor) • Sorting with ORDER BY (Alphabetically, by credits) • Aggregation with GROUP BY (Student count per course, average age) • Nested Subqueries for Complex Queries

```
-- Create Database
```

```
CREATE DATABASE StudentCourseDB;
```

```
-- Use Database
```

```
USE StudentCourseDB;
```

```
-- Create Student Table
```

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Email VARCHAR(100)
);
```

```
-- Create Instructor Table
```

```
CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50)
);
```

```
-- Create Course Table
```

```
CREATE TABLE Course (
```

```
CourseID INT PRIMARY KEY,  
CourseName VARCHAR(50),  
Credits INT,  
InstructorID INT,  
FOREIGN KEY (InstructorID)  
REFERENCES Instructor(InstructorID)  
);
```

-- Create Enrollment Table

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    FOREIGN KEY (StudentID)  
    REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID)  
    REFERENCES Course(CourseID)  
);
```

-- Insert Student Records

```
INSERT INTO Student VALUES  
(1, 'Arjun', 21, 'arjun@gmail.com'),  
(2, 'Rahul', 24, 'rahul@gmail.com'),  
(3, 'Sneha', 20, 'sneha@gmail.com'),  
(4, 'Priya', 23, 'priya@gmail.com');
```

-- Insert Instructor Records

```
INSERT INTO Instructor VALUES  
(101, 'Dr Sharma', 'Computer'),  
(102, 'Dr Mehta', 'IT');
```

-- Insert Course Records

```
INSERT INTO Course VALUES
```

```
(201, 'Database', 4, 101),
```

```
(202, 'Python', 3, 102),
```

```
(203, 'AI', 5, 101);
```

```
-- Insert Enrollment Records
```

```
INSERT INTO Enrollment VALUES
```

```
(1, 1, 201),
```

```
(2, 2, 201),
```

```
(3, 3, 202),
```

```
(4, 4, 203);
```

```
-----  
-- WHERE Clause  
-----
```

```
-- Students above age 21
```

```
SELECT * FROM Student
```

```
WHERE Age > 21;
```

```
-- Courses taught by Instructor 101
```

```
SELECT * FROM Course
```

```
WHERE InstructorID = 101;
```

```
-----  
-- ORDER BY  
-----
```

```
-- Students in alphabetical order
```

```
SELECT * FROM Student
```

```
ORDER BY Name ASC;
```


-- Courses sorted by credits

```
SELECT * FROM Course
ORDER BY Credits DESC;
```

-- GROUP BY

-- Student count per course

```
SELECT CourseID, COUNT(StudentID) AS TotalStudents
FROM Enrollment
GROUP BY CourseID;
```

-- Average student age

```
SELECT AVG(Age) AS AverageAge
FROM Student;
```

-- Nested Subqueries

-- Students enrolled in Database course

```
SELECT Name
FROM Student
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollment
    WHERE CourseID = (
        SELECT CourseID
        FROM Course
        WHERE CourseName = 'Database'
    )
)
```

```
);

-- Courses having credits greater than average credits
SELECT CourseName, Credits
FROM Course
WHERE Credits > (
    SELECT AVG(Credits)
    FROM Course
);
```

Q4) Multi-Table Joins

Develop a Customer Order Management System to Retrieve Data Using SQL Joins Objective: Learn inner and outer joins in SQL. Task: • INNER JOIN: Retrieve all customer orders (only customers who have placed orders). • LEFT JOIN: Retrieve all customers, including those who haven't placed any orders. • RIGHT JOIN: Retrieve all orders, including those placed by customers not in the Customers table. • FULL OUTER JOIN: Retrieve a complete list of customers and orders, including those that do not have a match in the other table.

```
-- Create Database
CREATE DATABASE CustomerOrderDB;

-- Use Database
USE CustomerOrderDB;

-- Create Customers Table
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    City VARCHAR(50)
);

-- Create Orders Table
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
```

```
CustomerID INT,  
Product VARCHAR(50),  
Amount INT  
);
```

```
-- Insert Records into Customers
```

```
INSERT INTO Customers VALUES
```

```
(1, 'Arjun', 'Pune'),  
(2, 'Rahul', 'Mumbai'),  
(3, 'Sneha', 'Delhi'),  
(4, 'Priya', 'Chennai');
```

```
-- Insert Records into Orders
```

```
INSERT INTO Orders VALUES
```

```
(101, 1, 'Laptop', 50000),  
(102, 2, 'Mobile', 20000),  
(103, 1, 'Keyboard', 2000),  
(104, 5, 'Mouse', 1000);
```

```
-----  
-- INNER JOIN  
-----
```

```
SELECT Customers.CustomerID,  
Customers.CustomerName,  
Orders.OrderID,  
Orders.Product  
FROM Customers  
INNER JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

```
-----
```

-- LEFT JOIN

```
SELECT Customers.CustomerID,  
Customers.CustomerName,  
Orders.OrderID,  
Orders.Product  
FROM Customers  
LEFT JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

-- RIGHT JOIN

```
SELECT Customers.CustomerID,  
Customers.CustomerName,  
Orders.OrderID,  
Orders.Product  
FROM Customers  
RIGHT JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

-- FULL OUTER JOIN

-- MySQL does not directly support FULL OUTER JOIN

-- So we use UNION of LEFT and RIGHT JOIN

```
SELECT Customers.CustomerID,  
Customers.CustomerName,  
Orders.OrderID,
```

```
Orders.Product
FROM Customers
LEFT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID
```

```
UNION
```

```
SELECT Customers.CustomerID,
Customers.CustomerName,
Orders.OrderID,
Orders.Product
FROM Customers
RIGHT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

Q5) PL/SQL

Procedures and Functions Design and Implement a Sales Management System with Stored Procedures and Functions for Efficient Data Handling Objective: Implement stored procedures and functions. Task: • Create a stored procedure to add data to a table. • Create a function to calculate and return total sales.

```
-- Create Database
```

```
CREATE DATABASE SalesDB;
```

```
-- Use Database
```

```
USE SalesDB;
```

```
-- Create Sales Table
```

```
CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Quantity INT,
    Price INT
);
```

```
-----  
-- Stored Procedure to Insert Data  
-----
```

```
DELIMITER //
```

```
CREATE PROCEDURE AddSale(  
    IN p_SaleID INT,  
    IN p_ProductName VARCHAR(50),  
    IN p_Quantity INT,  
    IN p_Price INT  
)  
BEGIN  
    INSERT INTO Sales  
    VALUES (p_SaleID, p_ProductName, p_Quantity, p_Price);  
END //
```

```
DELIMITER ;
```

```
-----  
-- Call Procedure  
-----
```

```
CALL AddSale(1, 'Laptop', 2, 50000);  
CALL AddSale(2, 'Mobile', 3, 20000);  
CALL AddSale(3, 'Keyboard', 5, 1000);
```

```
-----  
-- Display Table  
-----
```

```
SELECT * FROM Sales;
```

```
-----  
-- Function to Calculate Total Sales  
-----
```

```
DELIMITER //
```

```
CREATE FUNCTION TotalSales()
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE Total INT;
```

```
    SELECT SUM(Quantity * Price)
```

```
    INTO Total
```

```
    FROM Sales;
```

```
    RETURN Total;
```

```
END //
```

```
DELIMITER ;
```

```
-----  
-- Call Function  
-----
```

```
SELECT TotalSales() AS Total_Amount;
```

Q6) Triggers and Cursors

Develop an Employee Management System with Triggers for Audit Logging and Cursors for Efficient Data Processing Objective: Implement triggers and cursors in PL/SQL. Task: • Create a trigger on the Employees table that logs changes (INSERT, UPDATE, DELETE) into the Employee_Audit table. • Use a cursor to iterate through the Employees table and process salary increments based on certain conditions (e.g., increase salary by 10% for employees in the IT department)

-- Create Database

```
CREATE DATABASE EmployeeDB;
```

-- Use Database

```
USE EmployeeDB;
```

-- Create Employees Table

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Department VARCHAR(50),  
    Salary INT  
);
```

-- Create Employee_Audit Table

```
CREATE TABLE Employee_Audit (  
    AuditID INT AUTO_INCREMENT PRIMARY KEY,  
    EmpID INT,  
    ActionType VARCHAR(20),  
    ActionDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```



```
-----  
-- Insert Sample Records  
-----
```

```
INSERT INTO Employees VALUES  
(1, 'Arjun', 'IT', 50000),  
(2, 'Rahul', 'HR', 40000),  
(3, 'Sneha', 'IT', 60000);
```

```
-----  
-- Trigger for INSERT  
-----
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_insert  
AFTER INSERT ON Employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO Employee_Audit(EmpID, ActionType)  
    VALUES (NEW.EmpID, 'INSERT');  
END //
```

```
DELIMITER ;
```

```
-----  
-- Trigger for UPDATE  
-----
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_update
AFTER UPDATE ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Audit(EmpID, ActionType)
    VALUES (NEW.EmpID, 'UPDATE');
END //
```

```
DELIMITER ;
```

```
-----
-- Trigger for DELETE
-----
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_delete
AFTER DELETE ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Audit(EmpID, ActionType)
    VALUES (OLD.EmpID, 'DELETE');
END //
```

```
DELIMITER ;
```

```
-----
-- Test Triggers
-----
```

```
-- INSERT
INSERT INTO Employees VALUES
```

```
(4, 'Priya', 'Finance', 45000);
```

```
-- UPDATE
```

```
UPDATE Employees
```

```
SET Salary = 55000
```

```
WHERE EmpID = 1;
```

```
-- DELETE
```

```
DELETE FROM Employees
```

```
WHERE EmpID = 2;
```

```
-----  
-- Display Audit Table  
-----
```

```
SELECT * FROM Employee_Audit;
```

```
-----  
-- Cursor for Salary Increment  
-----
```

```
DELIMITER //
```

```
CREATE PROCEDURE IncreaseSalary()
```

```
BEGIN
```

```
    DECLARE done INT DEFAULT 0;
```

```
    DECLARE e_id INT;
```

```
    DECLARE emp_cursor CURSOR FOR
```

```
    SELECT EmpID
```

```
    FROM Employees
```

```
    WHERE Department = 'IT';
```

```
DECLARE CONTINUE HANDLER
FOR NOT FOUND SET done = 1;
```

```
OPEN emp_cursor;
```

```
read_loop: LOOP
```

```
    FETCH emp_cursor INTO e_id;
```

```
    IF done = 1 THEN
```

```
        LEAVE read_loop;
```

```
    END IF;
```

```
    UPDATE Employees
```

```
    SET Salary = Salary + (Salary * 0.10)
```

```
    WHERE EmpID = e_id;
```

```
END LOOP;
```

```
CLOSE emp_cursor;
```

```
END //
```

```
DELIMITER ;
```

```
-----
```

```
-- Call Procedure
```

```
-----
```

```
CALL IncreaseSalary();
```

```
-----  
-- Display Updated Employees Table  
-----
```

```
SELECT * FROM Employees;
```

Q7) NoSQL Database Introduction – MongoDB Aggregation and Indexing Objectives: Work with MongoDB for NoSQL data storage. Use MongoDB aggregation and indexing for data analysis. Task: • Create a MongoDB collection for user data. • Perform CRUD (Create, Read, Update, Delete) operations on the collection. • Write aggregation queries to calculate statistics (e.g., total sales, average age). • Create indexes and observe performance improvements.

```
// Create Database
```

```
use UserDB
```

```
//-----
```

```
// Create Collection and Insert Data
```

```
//-----
```

```
db.users.insertMany([
```

```
{
```

```
  name: "Arjun",
```

```
  age: 21,
```

```
  city: "Pune",
```

```
  sales: 5000
```

```
},
```

```
{
```

```
  name: "Rahul",
```

```
  age: 24,
```

```
  city: "Mumbai",
```

```
  sales: 7000
```

```
},
```

```
{
```

```
  name: "Sneha",
```

```
    age: 20,  
    city: "Delhi",  
    sales: 4000  
  }  
])  
  
//-----  
// READ Operation  
//-----  
  
// Display all documents  
db.users.find()  
  
// Find users with age greater than 21  
db.users.find({ age: { $gt: 21 } })  
  
//-----  
// UPDATE Operation  
//-----  
  
// Update sales of Arjun  
db.users.updateOne(  
  { name: "Arjun" },  
  { $set: { sales: 6000 } }  
)  
  
// Display updated data  
db.users.find()  
  
//-----  
// DELETE Operation  
//-----
```

```
// Delete Rahul record
```

```
db.users.deleteOne({ name: "Rahul" })
```

```
// Display data after delete
```

```
db.users.find()
```

```
//-----
```

```
// Aggregation Queries
```

```
//-----
```

```
// Calculate total sales
```

```
db.users.aggregate([  
  {  
    $group: {  
      _id: null,  
      totalSales: { $sum: "$sales" }  
    }  
  }  
])
```

```
// Calculate average age
```

```
db.users.aggregate([  
  {  
    $group: {  
      _id: null,  
      averageAge: { $avg: "$age" }  
    }  
  }  
])
```

```
//-----
```

```
// Create Index
//-----

// Create index on name
db.users.createIndex({ name: 1 })

// Create index on age
db.users.createIndex({ age: 1 })

//-----
// Check Query Performance
//-----

db.users.find({ name: "Arjun" }).explain("executionStats")
```

How to Execute

Step 1

Open MongoDB shell.

Command:

mongosh

Step 2

Paste commands one by one.

Step 3

Press Enter after every command.

Expected Output

find()

Displays all documents.

Aggregation

Shows:

totalSales
averageAge

Q8) Case Study – Social Media Data Processing using NoSQL Objective: Process unstructured social media data using NoSQL databases. Task: • Import a dataset with social media posts into MongoDB. • Write queries to filter and analyze data (e.g., find top trending hashtags).

```
// Create Database
```

```
use SocialMediaDB
```

```
//-----
```

```
// Create Collection and Import Dataset
```

```
//-----
```

```
db.posts.insertMany([
{
  user: "Arjun",
  post: "Learning MongoDB is fun",
  likes: 120,
  hashtags: ["#MongoDB", "#NoSQL", "#Database"]
},
{
  user: "Rahul",
  post: "AI is the future",
  likes: 200,
  hashtags: ["#AI", "#ML", "#Technology"]
},
{
  user: "Sneha",
  post: "Working on web development",
  likes: 150,
  hashtags: ["#WebDev", "#JavaScript", "#MongoDB"]
},

```

```
{
  user: "Priya",
  post: "Machine Learning project completed",
  likes: 180,
  hashtags: ["#ML", "#AI", "#Python"]
}
])
```

```
//-----
// Display All Data
//-----
```

```
db.posts.find()
```

```
//-----
// Filter Queries
//-----
```

```
// Find posts with likes greater than 150
db.posts.find({
  likes: { $gt: 150 }
})
```

```
// Find posts containing #AI hashtag
db.posts.find({
  hashtags: "#AI"
})
```

```
//-----
// Analyze Data using Aggregation
//-----
```

```
// Find total likes
db.posts.aggregate([
{
  $group: {
    _id: null,
    totalLikes: { $sum: "$likes" }
  }
}]
)
```

```
// Find average likes
db.posts.aggregate([
{
  $group: {
    _id: null,
    averageLikes: { $avg: "$likes" }
  }
}]
)
```

```
// Find top trending hashtags
db.posts.aggregate([
{
  $unwind: "$hashtags"
},
{
  $group: {
    _id: "$hashtags",
    count: { $sum: 1 }
  }
},
{

```

```
    $sort: { count: -1 }  
  }  
])  
  
//-----  
// Create Index  
//-----  
  
db.posts.createIndex({ hashtags: 1 })  
  
//-----  
// Check Performance  
//-----  
  
db.posts.find({  
  hashtags: "#AI"  
}).explain("executionStats")\
```

How to Execute

Step 1

Open MongoDB shell:

```
mongosh
```

Step 2

Paste commands one by one.

Step 3

Press Enter after each command.

Expected Output

find()

Displays all social media posts.

Aggregation

Displays:

totalLikes

averageLikes

top hashtags

Q9) MySQL Database Backup and Recovery Objective: To understand and practice the process of taking a backup of a MySQL database to ensure data security and disaster recovery

Step 1: Create Sample Database

```
CREATE DATABASE BackupDB;
```

```
USE BackupDB;
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

```
INSERT INTO Student VALUES  
(1, 'Arjun', 21),
```

```
(2, 'Rahul', 22),  
(3, 'Sneha', 20);
```

```
SELECT * FROM Student;
```

Step 2: Take Backup of Database

Open CMD and run:

```
mysqldump -u root -p BackupDB > backup.sql
```

Then:

- Enter MySQL password
 - Backup file backup.sql will be created
-

Step 3: Check Backup File

You can see:

```
backup.sql
```

inside your current folder.

Usually found in:

```
C:\Users\YourName
```

Step 4: Delete Database (For Recovery Demo)

Login MySQL:

```
mysql -u root -p
```

Then run:

```
DROP DATABASE BackupDB;
```

Step 5: Recover Database

Exit MySQL using:

```
exit
```

Now run recovery command in CMD:

```
mysql -u root -p BackupDB < backup.sql
```

Step 6: Verify Recovery

Login again:

```
mysql -u root -p
```

Then:

```
USE BackupDB;
```

```
SELECT * FROM Student;
```

You will see restored data.